OBJECT-ORIENTED PROGRAMMING · JAVA

Code Smells & Refactoring

Spot it. Name it. Fix it. Test it.

16 common mistakes in real code, and the refactoring that fixes each one

What is refactoring?

Changing the **structure** of code without changing what it **does**. Same inputs, same outputs, cleaner design.

Refactoring IS

- Small, safe, behaviour-preserving steps
- Renaming, extracting, moving, simplifying
- Done continuously, not once a year
- Backed by tests after **every** step

Refactoring is NOT

- Adding features
- Fixing bugs (separate step!)
- Rewriting from scratch
- Editing and hoping it still works

Code smell: a surface sign that the design has a deeper problem. It still works, but it is hard to read, change or test. (Martin Fowler, *Refactoring*)

How we'll work today

1 SPOT

Look at the code.
What feels wrong?

→

2 NAME

Which smell is it?
Naming it helps.

→

3 FIX

Apply the matching refactoring, in small steps.

→

4 TEST

Run the tests.
Same behaviour?

How each smell is presented

YOUR TURN Code only, no answer. Ask the class the questions on the right. Give them a minute.

THE FIX Before vs after, the refactoring's name, and why it is better.

Tip: the agenda is clickable. "Back to agenda" in every footer jumps back.

Agenda: 16 smells in 5 families

Click any smell to jump to it.

CONDITIONALS

- 01 Deeply nested if-else
- 02 Long if / switch chain on a type
- 13 Null-check pyramids

LOOPS & CALLS

- 03 Loop inside a loop (to search)
- 06 Repeated function calls
- 07 Long call chains ("train wreck")
- 15 Manual loops that should be streams

METHODS

- 04 Too many method parameters
- 05 Boolean flag parameters
- 09 Long method

CLASSES & DUPLICATION

- 08 Duplicate code
- 10 God class
- 16 Hard-coded dependencies

READABILITY

- 11 Magic numbers and strings
- 12 Poor naming
- 14 Dead and commented-out code

PRACTICE

- Quiz: name the smell
- Smell hunt (group exercise)
- Let the IDE do it
- Cheat sheet

#01 · CONDITIONALS YOUR TURN

What's wrong with this code?

```
1  void publish(User user, Document doc) {
2      if (user != null) {
3          if (user.isActive()) {
4              if (user.hasPermission("EDIT")) {
5                  if (doc.isValid()) {
6                      save(doc);
7                  } else {
8                      throw new InvalidDocException();
9                  }
10             }
11         }
12     }
13 }
```

ASK THE CLASS

1. How many levels deep is **save(doc)**?
2. We need one more rule: *"doc must be under 10 MB"*. Where does it go?

HINT

Look at the shape of the code before you read it.

#01 · CONDITIONALS THE FIX

Deeply nested if-else → Guard clauses (early return)

BEFORE

```
void publish(User user, Document doc) {
    if (user != null) {
        if (user.isActive()) {
            if (user.hasPermission("EDIT")) {
                if (doc.isValid()) {
                    save(doc);
                } else {
                    throw new InvalidDocException();
                }
            }
        }
    }
}
```

AFTER

```
void publish(User user, Document doc) {
    if (user == null) return;
    if (!user.isActive()) return;
    if (!user.hasPermission("EDIT")) return;
    if (!doc.isValid()) throw new InvalidDocException();

    save(doc);
}
```

WHY IT'S BETTER

Flat: every rule is one line you can read on its own.

The "happy path" sits at the bottom, not buried 5 levels deep.

Adding a rule = adding one line.

#02 · CONDITIONALS YOUR TURN

What's wrong with this code?

```
double fee(String type, int hours) {
    if (type.equals("CAR"))        return hours * 10;
    else if (type.equals("BIKE"))  return hours * 5;
    else if (type.equals("TRUCK")) return hours * 25;
    else if (type.equals("BUS"))   return hours * 30;
    else throw new IllegalArgumentException(type);
}
```

ASK THE CLASS

1. We now support **MOTORCYCLE**. How many places must change?
2. What happens if someone types **"car"** in lowercase?

HINT

Every new vehicle means editing this method again.

#02 · CONDITIONALS THE FIX

Long if / switch chain on a type → Replace Conditional with Polymorphism (enum / Strategy)

BEFORE

```
double fee(String type, int hours) {
    if (type.equals("CAR"))        return hours * 10;
    else if (type.equals("BIKE"))  return hours * 5;
    else if (type.equals("TRUCK")) return hours * 25;
    else if (type.equals("BUS"))   return hours * 30;
    else throw new IllegalArgumentException(type);
}
```

AFTER

```
enum Vehicle {
    CAR(10), BIKE(5), TRUCK(25), BUS(30);

    private final int ratePerHour;
    Vehicle(int rate) { this.ratePerHour = rate; }

    double fee(int hours) { return hours * ratePerHour; }
}

double fee = Vehicle.valueOf(type).fee(hours);
```

WHY IT'S BETTER

New vehicle = one new enum constant. No method edits.

The compiler catches typos; strings don't.

If behaviour (not just a number) differs, use the Strategy pattern.

#03 · LOOPS & CALLS YOUR TURN

What's wrong with this code?

```java
for (Order o : orders) {
    for (Customer c : customers) {
        if (o.getCustomerId() == c.getId()) {
            o.setCustomer(c);
        }
    }
}
// 10,000 x 10,000 = 100,000,000 checks
```

ASK THE CLASS

1. 10,000 orders and 10,000 customers: how many comparisons?
2. Does the inner loop stop once it finds the customer?

HINT

Count the work: outer size x inner size.

#03 · LOOPS & CALLS THE FIX

Loop inside a loop (to search) → Build a lookup Map first

BEFORE

```
for (Order o : orders) {
    for (Customer c : customers) {
        if (o.getCustomerId() == c.getId()) {
            o.setCustomer(c);
        }
    }
}
// 10,000 x 10,000 = 100,000,000 checks
```

AFTER

```
Map<Integer, Customer> byId = new HashMap<>();
for (Customer c : customers) {
    byId.put(c.getId(), c);
}

for (Order o : orders) {
    o.setCustomer(byId.get(o.getCustomerId()));
}
// 10,000 + 10,000 = 20,000 operations
```

WHY IT'S BETTER

O(n x m) becomes O(n + m): 100 million checks become 20 thousand.

Not every nested loop is wrong: a 2D grid needs one.

The smell is nested loops used to **search**.

#04 · METHODS YOUR TURN

What's wrong with this code?

```
User createUser(String firstName, String lastName,
                String email, String phone,
                String city, String country,
                int age, boolean active, boolean admin) {
    ...
}

createUser("Ana", "Lee", "ana@x.com", null,
           "Montreal", "Canada", 22, true, false);
```

ASK THE CLASS

1. Which argument is **active**? Which is **admin**?
2. What if someone swaps **email** and **phone**? Will it compile?

HINT

Try reading the call without looking at the signature.

#04 · METHODS THE FIX

Too many method parameters → Introduce Parameter Object / Builder

BEFORE

```
User createUser(String firstName, String lastName,
                String email, String phone,
                String city, String country,
                int age, boolean active, boolean admin) {
    ...
}

createUser("Ana", "Lee", "ana@x.com", null,
           "Montreal", "Canada", 22, true, false);
```

AFTER

```
record Address(String city, String country) {}

User user = User.builder()
        .firstName("Ana")
        .lastName("Lee")
        .email("ana@x.com")
        .address(new Address("Montreal", "Canada"))
        .age(22)
        .active(true)
        .build();
```

WHY IT'S BETTER

Every value is named at the call site.

Optional fields (phone) are simply left out.

Data that travels together (city + country) becomes its own type.

#05 · METHODS YOUR TURN

What's wrong with this code?

```
1  void export(Report r, boolean pdf, boolean compress) {
2      if (pdf) {
3          writePdf(r);
4      } else {
5          writeHtml(r);
6      }
7      if (compress) zip(r);
8  }
9
10 export(report, true, false);   // ???
```

ASK THE CLASS

1. Read **export(report, true, false)** out loud. What does it do?
2. How many different behaviours hide inside this one method?

HINT

A boolean argument usually means the method does two jobs.

#05 · METHODS THE FIX

Boolean flag parameters → Remove Flag Argument (split into named methods)

BEFORE

```
void export(Report r, boolean pdf, boolean compress) {
    if (pdf) {
        writePdf(r);
    } else {
        writeHtml(r);
    }
    if (compress) zip(r);
}

export(report, true, false);   // ???
```

AFTER

```
void exportAsPdf(Report r)  { writePdf(r); }
void exportAsHtml(Report r) { writeHtml(r); }

exportAsPdf(report);
exportAsHtml(report);

// If the options keep growing, use enums instead:
export(report, Format.PDF, Compression.NONE);
```

WHY IT'S BETTER

The method name now says exactly what happens.

Each method does one thing and is easy to test.

Enums beat booleans when there are more than two choices.

#06 · LOOPS & CALLS YOUR TURN

What's wrong with this code?

```
1 if (repo.findUser(id).isActive()
2         && repo.findUser(id).getAge() >= 18) {
3     mailer.send(repo.findUser(id).getEmail(),
4         "Welcome " + repo.findUser(id).getName());
5 }
```

ASK THE CLASS

1. How many database queries does this run?
2. What if the user record changes between the first and last call?

HINT

Count how many times findUser(id) appears.

#06 · LOOPS & CALLS THE FIX

Repeated function calls → Extract Variable (call once, reuse)

BEFORE

```
if (repo.findUser(id).isActive()
        && repo.findUser(id).getAge() >= 18) {
    mailer.send(repo.findUser(id).getEmail(),
        "Welcome " + repo.findUser(id).getName());
}
```

AFTER

```
User user = repo.findUser(id);

if (user.isActive() && user.getAge() >= 18) {
    mailer.send(user.getEmail(),
                "Welcome " + user.getName());
}
```

WHY IT'S BETTER

4 queries become 1: faster and cheaper.

All checks use the same, consistent data.

Shorter lines, and the variable name explains the value.

#07 · LOOPS & CALLS YOUR TURN

What's wrong with this code?

```
String city = order.getCustomer()
                   .getAddress()
                   .getCity()
                   .getName();
```

ASK THE CLASS

1. How many classes does this one line depend on?
2. What happens if **getAddress()** returns null?

HINT

Count the dots.

#07 · LOOPS & CALLS THE FIX

Long call chains ("train wreck") → Hide Delegate (Law of Demeter)

BEFORE

```
String city = order.getCustomer()
                   .getAddress()
                   .getCity()
                   .getName();
```

AFTER

```
// In Order
String getShippingCityName() {
    return customer.getCityName();
}

// In Customer
String getCityName() {
    return address.getCityName();
}

// Caller
String city = order.getShippingCityName();
```

WHY IT'S BETTER

"Only talk to your immediate friends."

If Address changes, only Customer is affected, not every caller.

Note: fluent APIs like streams and builders are fine: they return the same kind of object.

#08 · CLASSES & DUPLICATION YOUR TURN

What's wrong with this code?

```java
double studentPrice(double base) {
    double discounted = base * 0.80;
    double tax = discounted * 0.14975;
    return Math.round((discounted + tax) * 100) / 100.0;
}

double seniorPrice(double base) {
    double discounted = base * 0.70;
    double tax = discounted * 0.14975;
    return Math.round((discounted + tax) * 100) / 100.0;
}
```

ASK THE CLASS

1. The tax rate changes tomorrow. How many places do you edit?
2. What happens if you forget one of them?

HINT

Spot the two lines that differ. Everything else is copied.

#08 · CLASSES & DUPLICATION THE FIX

Duplicate code → Extract Method (Don't Repeat Yourself)

BEFORE

```
double studentPrice(double base) {
    double discounted = base * 0.80;
    double tax = discounted * 0.14975;
    return Math.round((discounted + tax) * 100) / 100.0;
}

double seniorPrice(double base) {
    double discounted = base * 0.70;
    double tax = discounted * 0.14975;
    return Math.round((discounted + tax) * 100) / 100.0;
}
```

AFTER

```
private static final double TAX_RATE = 0.14975;

double studentPrice(double base) {
    return priceAfter(base, 0.80);
}
double seniorPrice(double base) {
    return priceAfter(base, 0.70);
}

private double priceAfter(double base, double rate) {
    double discounted = base * rate;
    double total = discounted * (1 + TAX_RATE);
    return Math.round(total * 100) / 100.0;
}
```

WHY IT'S BETTER

The shared logic lives in exactly one place.

Only the part that varies (the rate) is passed in.

Rule of three: write it, wince at the copy, refactor on the third.

#09 · METHODS YOUR TURN

What's wrong with this code?

```
1  void placeOrder(Order order) {
2      // 1. validate
3      if (order.getItems().isEmpty()) throw new BadOrder();
4      if (order.getCustomer() == null) throw new BadOrder();
5      // 2. calculate total
6      double total = 0;
7      for (Item i : order.getItems()) total += i.cost();
8      if (order.hasCoupon()) total *= 0.9;
9      // 3. save
10     db.insert("orders", order.getId(), total);
11     // 4. notify
12     mailer.send(order.getEmail(), "Total: $" + total);
13     // ... 150 more lines ...
14 }
```

ASK THE CLASS

1. The comments look like section headings. What are they telling you?
2. How would you unit-test only the total calculation?

HINT

Each comment block could be its own method.

#09 · METHODS THE FIX

Long method → Extract Method

BEFORE

```
void placeOrder(Order order) {
    // 1. validate
    if (order.getItems().isEmpty()) throw new BadOrder();
    if (order.getCustomer() == null) throw new BadOrder();
    // 2. calculate total
    double total = 0;
    for (Item i : order.getItems()) total += i.cost();
    if (order.hasCoupon()) total *= 0.9;
    // 3. save
    db.insert("orders", order.getId(), total);
    // 4. notify
    mailer.send(order.getEmail(), "Total: $" + total);
    // ... 150 more lines ...
}
```

AFTER

```
void placeOrder(Order order) {
    validate(order);
    double total = calculateTotal(order);
    save(order, total);
    sendConfirmation(order, total);
}

private void validate(Order order)           { ... }
private double calculateTotal(Order order)   { ... }
private void save(Order order, double total) { ... }
private void sendConfirmation(Order o, double t) { ... }
```

WHY IT'S BETTER

The top method now reads like a table of contents.

Each piece can be tested and reused on its own.

If you need a comment to explain a block, extract it and use the comment as the name.

#10 · CLASSES & DUPLICATION

YOUR TURN

What's wrong with this code?

```
class UserManager {
    void login(String user, String pass)   { ... }
    void logout(User u)                    { ... }
    void chargeCard(User u, double amount) { ... }
    void refund(Payment p)                 { ... }
    void sendWelcomeEmail(User u)          { ... }
    void sendInvoice(User u, Invoice i)    { ... }
    Report monthlyReport()                 { ... }
    void writeLog(String msg)              { ... }
    // 2,400 lines, 60 methods, 25 fields
}
```

ASK THE CLASS

1. Finish the sentence: *"UserManager is responsible for ..."*
2. Can you say it without using the word **and**?

HINT

Group the methods by topic. Each group is a class waiting to be born.

#10 · CLASSES & DUPLICATION THE FIX

God class → Extract Class (Single Responsibility Principle)

BEFORE

```
class UserManager {
    void login(String user, String pass)   { ... }
    void logout(User u)                    { ... }
    void chargeCard(User u, double amount) { ... }
    void refund(Payment p)                 { ... }
    void sendWelcomeEmail(User u)          { ... }
    void sendInvoice(User u, Invoice i)    { ... }
    Report monthlyReport()                 { ... }
    void writeLog(String msg)              { ... }
    // 2,400 lines, 60 methods, 25 fields
}
```

AFTER

```
class AuthService    { void login(...);  void logout(...); }
class PaymentService { void charge(...); void refund(...); }
class EmailService   { void sendWelcome(...);
                       void sendInvoice(...); }
class ReportService  { Report monthly(); }
class AppLogger      { void write(String msg); }
```

WHY IT'S BETTER

Each class has one reason to change.

Smaller classes are easier to read, test and assign to teammates.

Fewer merge conflicts: people stop editing the same giant file.

#11 · READABILITY YOUR TURN

What's wrong with this code?

```
1 if (order.getStatus() == 3) {
2     double total = price * 1.14975;
3     if (weight > 30) total += 15;
4     sleep(86400000);
5 }
```

ASK THE CLASS

1. What is **3**? What is **1.14975**? What is **86400000**?
2. If the heavy-parcel limit changes, how do you find every use of 30?

HINT

Any unexplained literal is a magic number.

#11 · READABILITY THE FIX

Magic numbers and strings → Replace with named constants / enums

BEFORE

```
if (order.getStatus() == 3) {
    double total = price * 1.14975;
    if (weight > 30) total += 15;
    sleep(86400000);
}
```

AFTER

```
static final double TAX_MULTIPLIER = 1.14975; // GST + QST
static final int HEAVY_PARCEL_KG = 30;
static final double HEAVY_SURCHARGE = 15.0;
static final Duration ONE_DAY = Duration.ofDays(1);

if (order.getStatus() == OrderStatus.SHIPPED) {
    double total = price * TAX_MULTIPLIER;
    if (weight > HEAVY_PARCEL_KG) total += HEAVY_SURCHARGE;
    sleep(ONE_DAY.toMillis());
}
```

WHY IT'S BETTER

The code now explains itself.

Each value is changed in one place.

Enums make invalid states (status = 42) impossible.

#12 · READABILITY YOUR TURN

What's wrong with this code?

```
1 int d;
2 List<String> lst = new ArrayList<>();
3 boolean flag = false;
4
5 void proc(int x) {
6     Data tmp = get(x);
7     if (tmp.f) lst.add(tmp.s);
8 }
```

ASK THE CLASS

1. Without guessing: what does **proc** do?
2. What is **d**? Days? Distance? Discount?

HINT

If you need to read the body to understand the name, the name failed.

#12 · READABILITY THE FIX

Poor naming → Rename (variables, methods, classes)

BEFORE

```
int d;
List<String> lst = new ArrayList<>();
boolean flag = false;

void proc(int x) {
    Data tmp = get(x);
    if (tmp.f) lst.add(tmp.s);
}
```

AFTER

```
int daysSinceLastLogin;
List<String> overdueCustomerEmails = new ArrayList<>();
boolean isReportReady = false;

void collectOverdueEmail(int customerId) {
    Customer customer = findCustomer(customerId);
    if (customer.isOverdue()) {
        overdueCustomerEmails.add(customer.getEmail());
    }
}
```

WHY IT'S BETTER

Good names remove the need for comments.

Booleans read like questions: isReady, hasAccess.

In IntelliJ, Shift+F6 renames safely across the whole project.

#13 · CONDITIONALS YOUR TURN

What's wrong with this code?

```
String city = "Unknown";
if (user != null) {
    Address a = user.getAddress();
    if (a != null) {
        City c = a.getCity();
        if (c != null && c.getName() != null) {
            city = c.getName();
        }
    }
}
```

ASK THE CLASS

1. How many null checks are there?
2. Which of these values should be allowed to be null in the first place?

HINT

This is smell #1 (nesting) caused by nulls.

#13 · CONDITIONALS THE FIX

Null-check pyramids → Optional (or don't return null at all)

BEFORE

```
String city = "Unknown";
if (user != null) {
    Address a = user.getAddress();
    if (a != null) {
        City c = a.getCity();
        if (c != null && c.getName() != null) {
            city = c.getName();
        }
    }
}
```

AFTER

```
String city = Optional.ofNullable(user)
        .map(User::getAddress)
        .map(Address::getCity)
        .map(City::getName)
        .orElse("Unknown");
```

WHY IT'S BETTER

One readable pipeline instead of four nested checks.

The default value is stated once, at the end.

Better still: getters return Optional, or a Null Object, so null never escapes.

#14 · READABILITY YOUR TURN

What's wrong with this code?

```
1  double calculateTotal(Cart cart) {
2      double total = 0;
3      int count = 0;                          // never used
4      // double oldTotal = legacyCalc(cart);
5      // if (USE_OLD) return oldTotal;
6      for (Item i : cart.items()) total += i.price();
7      return total;
8  }
9
10 private double legacyCalc(Cart c) { ... }   // no callers
```

ASK THE CLASS

1. Which lines actually run?
2. Why are people afraid to delete commented-out code?

HINT

Unused variables, unused methods, and code in comments.

#14 · READABILITY THE FIX

Dead and commented-out code → Delete it (Git remembers)

BEFORE

```
double calculateTotal(Cart cart) {
    double total = 0;
    int count = 0;                          // never used
    // double oldTotal = legacyCalc(cart);
    // if (USE_OLD) return oldTotal;
    for (Item i : cart.items()) total += i.price();
    return total;
}

private double legacyCalc(Cart c) { ... }   // no callers
```

AFTER

```
double calculateTotal(Cart cart) {
    double total = 0;
    for (Item i : cart.items()) total += i.price();
    return total;
}
```

WHY IT'S BETTER

Less code to read, maintain and misunderstand.

Version control keeps the history if you ever need it back.

IDEs grey out unused code. Treat grey as "delete me".

#15 · LOOPS & CALLS YOUR TURN

What's wrong with this code?

```
List<String> names = new ArrayList<>();
for (User u : users) {
    if (u.isActive()) {
        String n = u.getName().toUpperCase();
        if (!names.contains(n)) {
            names.add(n);
        }
    }
}
Collections.sort(names);
```

ASK THE CLASS

1. Describe what this loop does in one English sentence.
2. Is there a hidden performance problem? (Hint: contains)

HINT

filter, map, remove duplicates, sort.

#15 · LOOPS & CALLS THE FIX

Manual loops that should be streams → Replace Loop with Pipeline

BEFORE

```
List<String> names = new ArrayList<>();
for (User u : users) {
    if (u.isActive()) {
        String n = u.getName().toUpperCase();
        if (!names.contains(n)) {
            names.add(n);
        }
    }
}
Collections.sort(names);
```

AFTER

```
List<String> names = users.stream()
        .filter(User::isActive)
        .map(u -> u.getName().toUpperCase())
        .distinct()
        .sorted()
        .toList();
```

WHY IT'S BETTER

The code reads like the English sentence you just said.

No temporary list to manage by hand.

Bonus: contains() inside a loop was $O(n^2)$; distinct() is not.

#16 · CLASSES & DUPLICATION YOUR TURN

What's wrong with this code?

```
1  class OrderService {
2      private final EmailSender sender = new GmailSender();
3      private final OrderRepo repo = new MySqlOrderRepo();
4
5      void place(Order o) {
6          repo.save(o);
7          sender.send(o.getEmail(), "Order received");
8      }
9  }
10 // Every test sends a REAL email and needs a REAL database
```

ASK THE CLASS

1. How do you unit-test **place()** without sending a real email?
2. What if we move from MySQL to PostgreSQL?

HINT

Look for the word new inside the class.

#16 · CLASSES & DUPLICATION THE FIX

Hard-coded dependencies → Dependency Injection (pass dependencies in)

BEFORE

```
class OrderService {
    private final EmailSender sender = new GmailSender();
    private final OrderRepo repo = new MySqlOrderRepo();

    void place(Order o) {
        repo.save(o);
        sender.send(o.getEmail(), "Order received");
    }
}
// Every test sends a REAL email and needs a REAL database
```

AFTER

```
class OrderService {
    private final EmailSender sender;
    private final OrderRepo repo;

    OrderService(EmailSender sender, OrderRepo repo) {
        this.sender = sender;
        this.repo = repo;
    }
    void place(Order o) { ... }   // unchanged
}

// In a test:
new OrderService(new FakeSender(), new InMemoryOrderRepo());
```

WHY IT'S BETTER

The class depends on interfaces, not concrete classes.

Tests can pass fakes: fast, no network, no database.

Usually the first refactoring on legacy code: it makes the rest testable.

QUIZ

Name the smell. You have 2 minutes.

A

```
calc(order, true, true, false);
```

B

```
if (user.getRole() == 2) { ... }
```

C

```
cfg.getDb().getPool().size();
```

D

```
for (Student a : list)
  for (Student b : list)
    if (a.getId() == b.getId()) ...
```

E

```
class AppManager {
  // 3,000 lines, 80 methods
}
```

F

```
int t; String s2;
void doIt(int x) { ... }
```

ANSWERS

Name the smell: answers

A

```
calc(order, true, true, false);
```

Boolean flag parameters (#05)

B

```
if (user.getRole() == 2) { ... }
```

Magic number (#11)

C

```
cfg.getDb().getPool().size();
```

Long call chain (#07)

D

```
for (Student a : list)
  for (Student b : list)
    if (a.getId() == b.getId()) ...
```

Loop inside a loop (#03)

E

```
class AppManager {
  // 3,000 lines, 80 methods
}
```

God class (#10)

F

```
int t; String s2;
void doIt(int x) { ... }
```

Poor naming (#12)

GROUP EXERCISE · 3 MINUTES

Smell hunt: how many can your group find?

```java
public double proc(String t, double a, int q, boolean b, boolean c) {
    double r = 0;
    if (t != null) {
        if (t.equals("A")) {
            r = a * q * 0.9;
        } else if (t.equals("B")) {
            r = a * q * 0.8;
        } else if (t.equals("C")) {
            r = a * q * 0.7;
        }
    }
    if (b) {
        r = r + r * 0.14975;
    }
    if (c) {
        System.out.println("Total: " + r);
    }
    // double old = a * q;
    return r;
}
```

RULES

- Work in pairs or groups of 3.
- Write down every smell you see and the line number.
- For each one, name the refactoring that fixes it.
- The group with the most correct answers explains their fix.

THERE ARE AT LEAST

8 smells hiding here

SMELL HUNT · ANSWERS

Smell hunt: what we found

```
1  public double proc(String t, double a, int q, boolean b, boolean c) {
2      double r = 0;
2      if (t != null) {
3          if (t.equals("A")) {
4              r = a * q * 0.9;
6          } else if (t.equals("B")) {
7              r = a * q * 0.8;
8          } else if (t.equals("C")) {
9              r = a * q * 0.7;
10         }
11     }
5      if (b) {
6          r = r + r * 0.14975;
14     }
15     if (c) {
7          System.out.println("Total: " + r);
17     }
8      // double old = a * q;
19     return r;
20 }
```

1. **Poor naming + too many params + boolean flags**
   → Rename, Parameter Object, split methods
2. **Nested if (null check wraps everything)**
   → Guard clause
3. **If-else chain on a type string**
   → Enum / polymorphism
4. **Magic numbers 0.9 / 0.8 / 0.7 + duplicated a * q**
   → Constants in the enum, Extract Method
5. **Boolean flag b changes behaviour**
   → Separate withTax() method
6. **Magic number 0.14975**
   → TAX_RATE constant
7. **Calculating AND printing in one method**
   → Return a value, let the caller print
8. **Commented-out code**
   → Delete it

SMELL HUNT · REFACTORED

One possible clean version

```java
static final double TAX_RATE = 0.14975; // GST + QST

enum Tier {
    GOLD(0.90), SILVER(0.80), BRONZE(0.70);

    private final double priceMultiplier;
    Tier(double m) { this.priceMultiplier = m; }

    double apply(double amount) {
        return amount * priceMultiplier;
    }
}

double subtotal(Tier tier, double unitPrice, int quantity) {
    return tier.apply(unitPrice * quantity);
}

double withTax(double amount) {
    return amount * (1 + TAX_RATE);
}

// The caller decides what to print:
double total = withTax(subtotal(Tier.GOLD, 19.99, 3));
System.out.println("Total: " + total);
```

DISCUSS

- Is this **exactly** the same behaviour? What did **t == null** return before, and what happens now?
- Which version would you rather change next month?
- What tests would you write **before** doing this refactoring?

Let the IDE do it: IntelliJ IDEA shortcuts

Automated refactorings update every usage for you, so they are much safer than editing by hand.

Refactoring	Windows / Linux	macOS
Refactor This (menu of everything)	`Ctrl + Alt + Shift + T`	`Ctrl + T`
Rename	`Shift + F6`	`Shift + F6`
Extract Method	`Ctrl + Alt + M`	`Cmd + Opt + M`
Extract Variable	`Ctrl + Alt + V`	`Cmd + Opt + V`
Extract Constant	`Ctrl + Alt + C`	`Cmd + Opt + C`
Extract Parameter	`Ctrl + Alt + P`	`Cmd + Opt + P`
Inline (undo an extract)	`Ctrl + Alt + N`	`Cmd + Opt + N`
Change Signature	`Ctrl + F6`	`Cmd + F6`

TOOLS THAT FIND SMELLS

- **SonarQube for IDE** (free plugin)
- **PMD** + CPD (copy-paste)
- **Checkstyle**
- **JDeodorant** (Eclipse)

Safety net:

- **JUnit** tests
- **JaCoCo** coverage

Cheat sheet: smell → refactoring

01 **Deeply nested if-else**
→ Guard clauses (early return)

02 **Long if / switch chain on a type**
→ Replace Conditional with Polymorphism (enum / Strategy)

03 **Loop inside a loop (to search)**
→ Build a lookup Map first

04 **Too many method parameters**
→ Introduce Parameter Object / Builder

05 **Boolean flag parameters**
→ Remove Flag Argument (split into named methods)

06 **Repeated function calls**
→ Extract Variable (call once, reuse)

07 **Long call chains ("train wreck")**
→ Hide Delegate (Law of Demeter)

08 **Duplicate code**
→ Extract Method (Don't Repeat Yourself)

09 **Long method**
→ Extract Method

10 **God class**
→ Extract Class (Single Responsibility Principle)

11 **Magic numbers and strings**
→ Replace with named constants / enums

12 **Poor naming**
→ Rename (variables, methods, classes)

13 **Null-check pyramids**
→ Optional (or don't return null at all)

14 **Dead and commented-out code**
→ Delete it (Git remembers)

15 **Manual loops that should be streams**
→ Replace Loop with Pipeline

16 **Hard-coded dependencies**
→ Dependency Injection (pass dependencies in)

THE GOLDEN RULE

No tests, no refactoring.

Without tests you can't prove the behaviour stayed the same. You're not refactoring, you're just changing code and hoping.

Small step → run tests → commit → repeat

Further reading: Martin Fowler, Refactoring (2nd ed.) · Joshua Kerievsky, Refactoring to Patterns · refactoring.guru